ADVANCED PROGRAMMING | ASSIGNMENT 1

Student Marks Management API

An ASP.NET Core service with PostgreSQL persistence, JWT role rules, graded reports and repeatable API testing.

Asif Baig | Registration: **23F-0053**

FAST NUCES, Chiniot-Faisalabad Campus | 10 November 2026

Repository:
https://github.com/asifbaig7034/23F-0053_ASIF_Assignment1

Architecture and responsibilities

Endpoints	Services	EF Core	PostgreSQL
HTTP, DTO validation and access policies	Interfaces and application operations	Scoped DbContext and migrations	Tables, indexes and constraints

JWT authentication runs before protected endpoints. Teacher policies guard writes and administration. Student access to a record or transcript checks the linked `studentId` claim.

Runtime and tooling

.NET 10 / ASP.NET Core
PostgreSQL 17.11
EF Core + Npgsql
Swagger UI and Postman

Verified implementation results

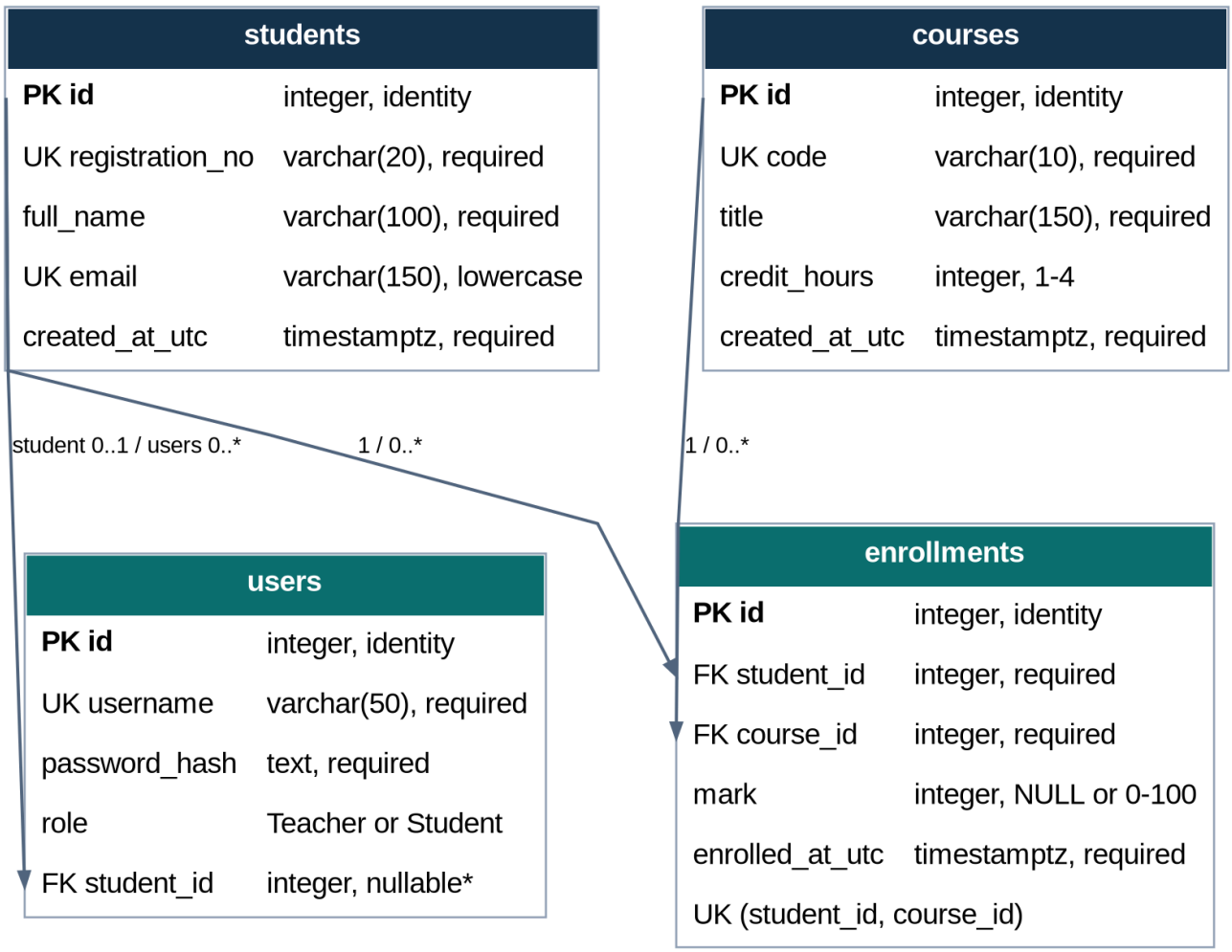
118 / 118 Postman tests passed
107 / 107 smoke checks passed
Smoke process exit code: **0**
HTTPS redirect and CORS checks passed

Production practices

Responses use camelCase DTOs and UTC timestamps. Validation problems and other errors use Problem Details. Read-only queries use AsNoTracking and projections; database calls use async methods and cancellation tokens. ILogger records operations without logging passwords or JWTs.

PASSED - fresh database, clean GitHub clone, two smoke runs and full Postman run.

Schema and integrity



PK = primary key; FK = foreign key; UK = unique index. Cardinalities show parent/child participation. EF's __EFMigrationsHistory table is infrastructure and is omitted.

Database guarantees

- Unique registration numbers, lowercase emails, course codes and usernames.
- One enrollment per student/course pair.
- Credit hours 1-4; mark NULL or integer 0-100.
- All three foreign keys use ON DELETE RESTRICT.

Accounts and deletion

A Student-role user must have a student link. That link is nullable for other roles. The FK index is non-unique, so the schema permits multiple accounts linked to a student.

Deleting an enrolled student/course, or a student with a linked account, returns 409.

> **Concurrency:** inserts rely on PostgreSQL unique indexes. SQLSTATE 23505 is translated to 409 instead of a check-then-insert race. Existing migrations are the authoritative setup; schema.sql is the actual schema-only export.

Authentication and configuration

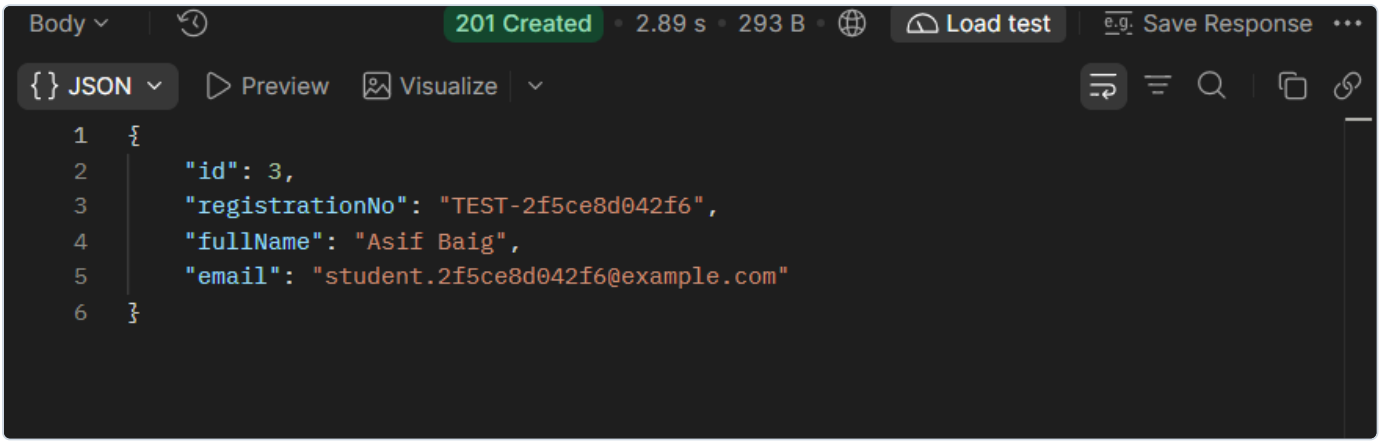
Passwords use ASP.NET Core PasswordHasher. Signed JWTs include expiry and role claims. Development credentials and the signing key come from user-secrets; environment variables support deployment. CORS permits configured origins. HTTP redirects to HTTPS; interactive Swagger is available in Development.

Grades and input validation

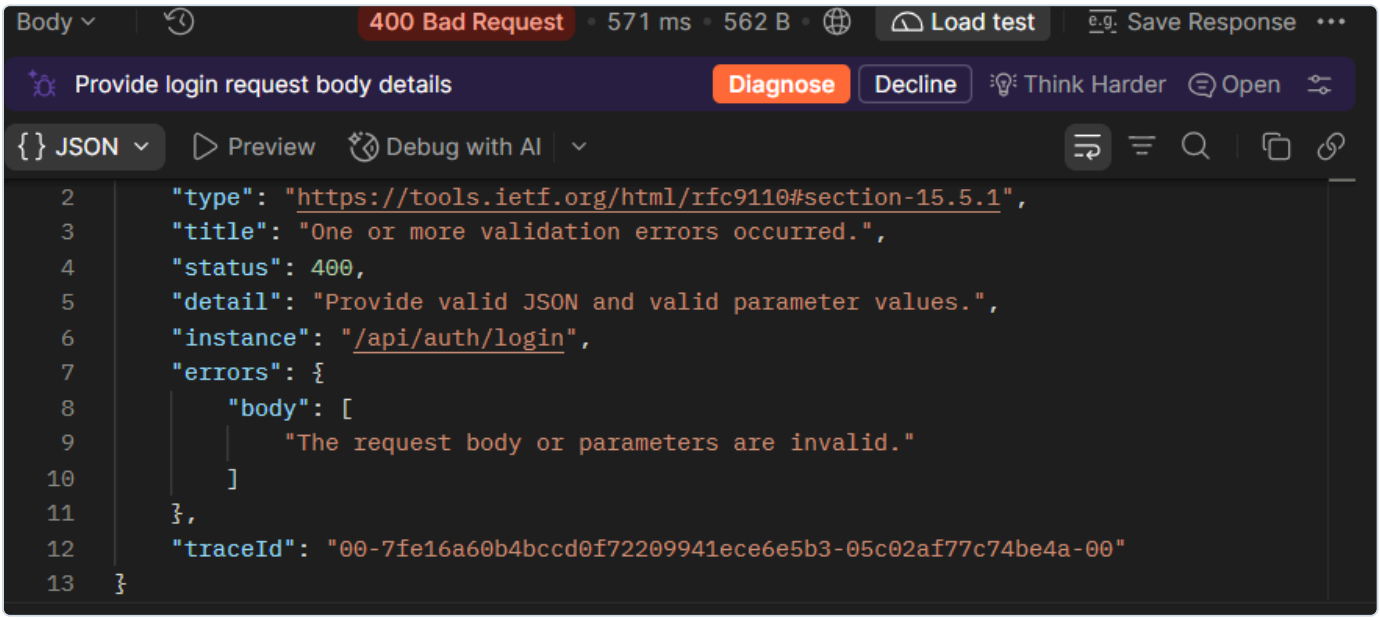
> **Weighted average:** sum(mark x credit hours) / sum(credit hours), using graded courses only.
> **Seed example:** (80 x 3 + 60 x 2) / (3 + 2) = **72**.

NULL means ungraded and is excluded from averages. Zero is a graded mark. Empty/ungraded averages are NULL. A course passes at the configured threshold (50). CS401 marks 80 and 70 give a course average of 75.

201 Created | POST /api/students - successful DTO response



400 Bad Request | POST /api/auth/login - missing/invalid request body

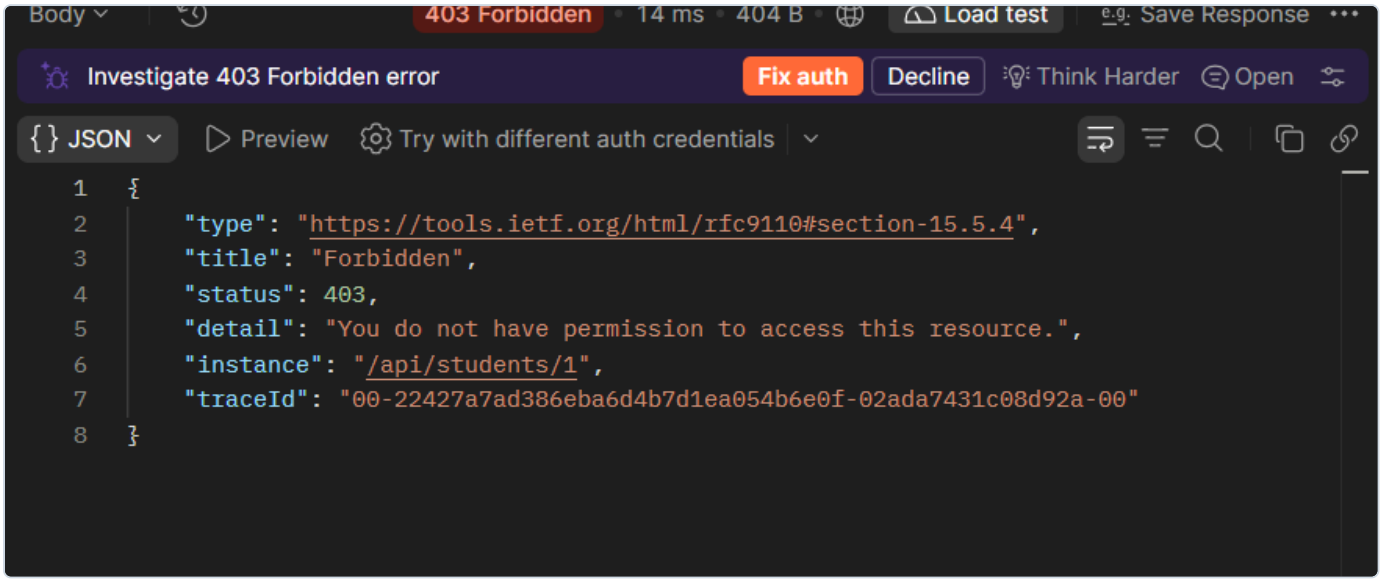


These screenshots were captured during implementation. The original complete screenshots are included in docs/screenshots; only the relevant response areas are shown here.

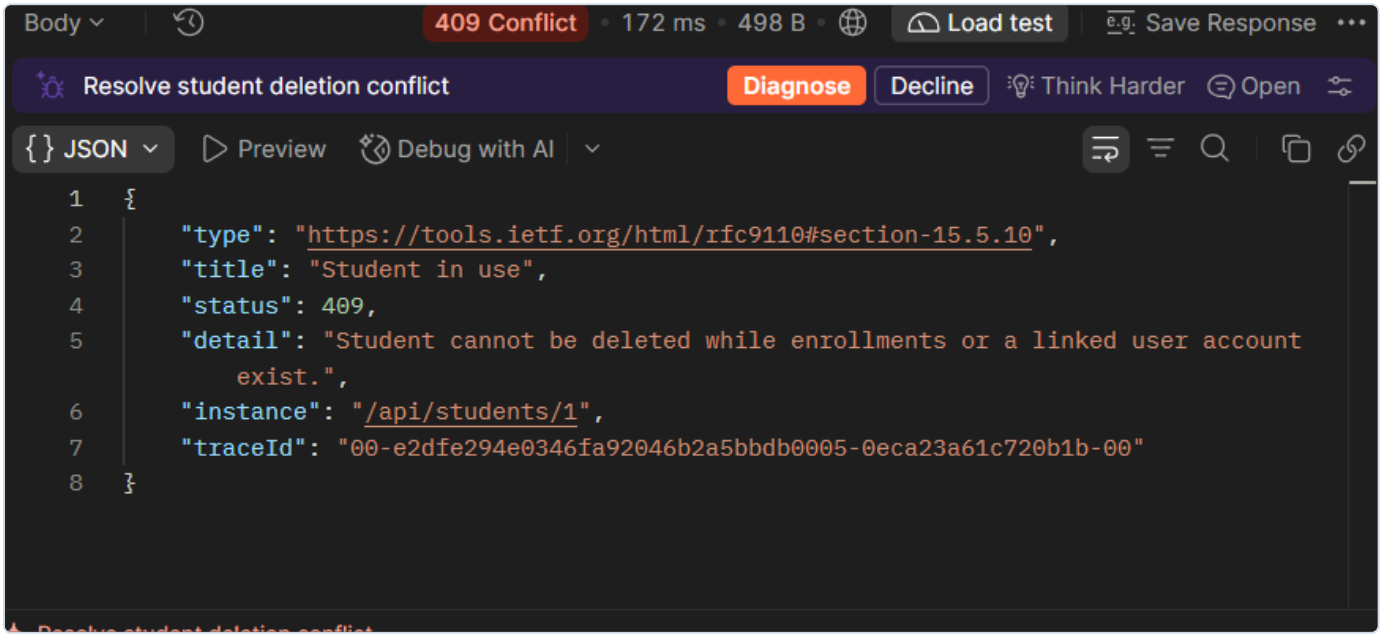
Expected error behavior

401 requires valid authentication. 403 means the caller lacks permission. 404 identifies missing resources. 409 reports a conflict with existing data. Error bodies consistently contain status, title and detail.

403 Forbidden | Student attempts DELETE /api/students/1



409 Conflict | Teacher attempts to delete an enrolled student



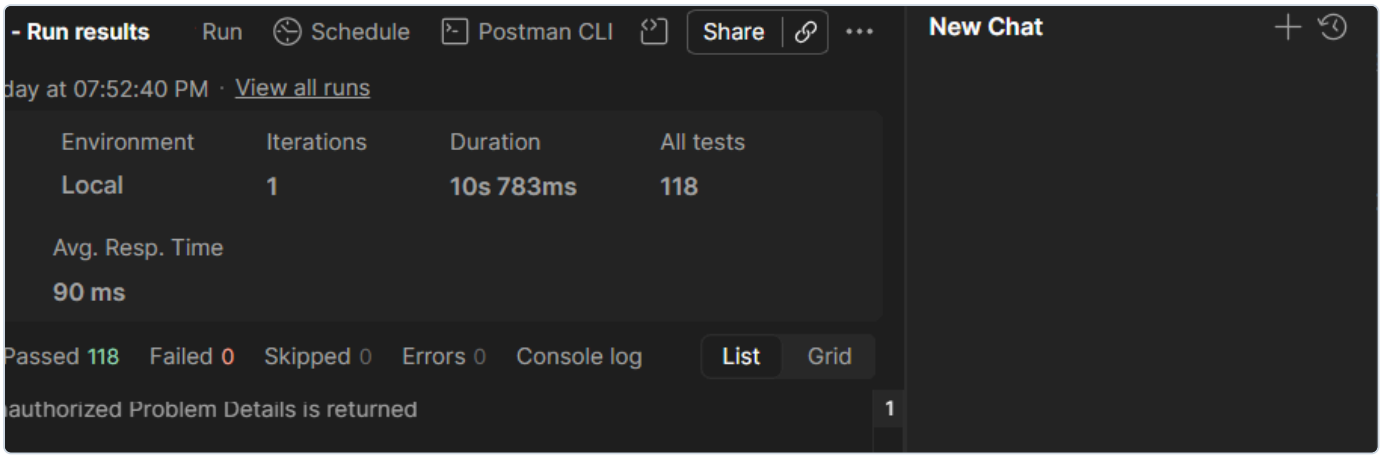
Issues resolved during implementation

PostgreSQL tools were located through their installation bin path. Database credentials were moved through user-secrets. Postman login scripts were corrected to keep Teacher and Student tokens separate; stale collection URLs, methods and ordering were corrected before the full passing run.

Repeatable verification

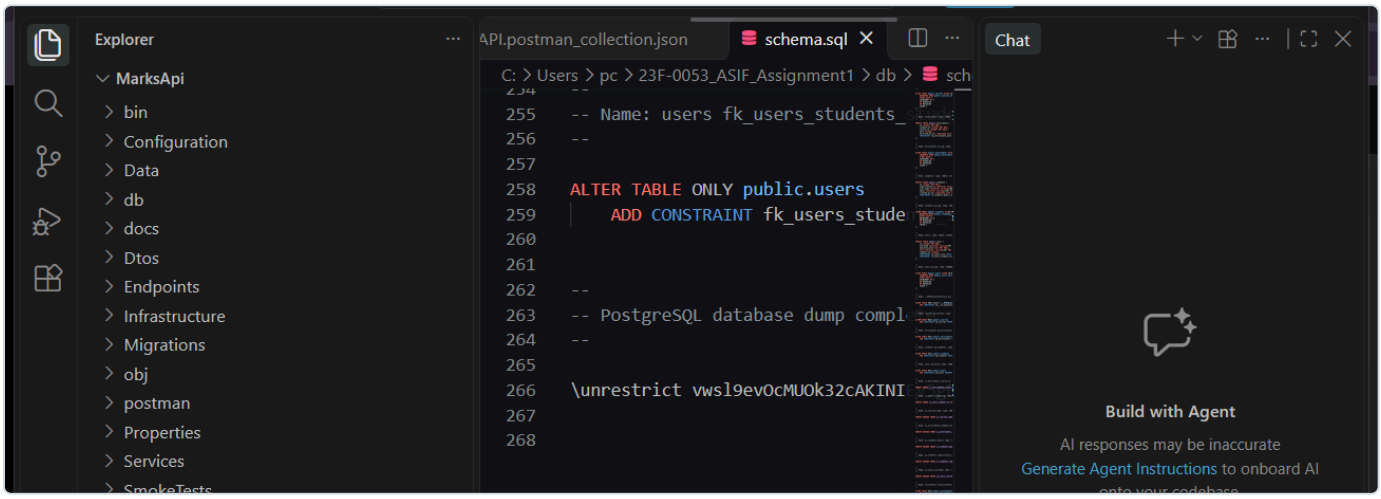
The collection contains **59 requests, 118 tests and 32 negative cases**. Auth runs first; create requests save generated IDs; CRUD/enrollment cleanup precedes the final negative cases. The exported environment has blank token values.

Postman Collection Runner | 118 passed, 0 failed, 0 errors



Startup smoke module

The opt-in --smoke-test mode starts the real HTTP server, uses PostgreSQL, checks successes/errors and report edge cases, removes SMOKE-prefixed fixtures and exits 0 only on success. Development credentials come from configuration. Both recorded runs passed 107 checks.



PASSED - fresh database, clean GitHub clone, two smoke runs and full Postman run.

The final helper records the clean Git revision, separate verification database, seed/cleanup row counts, two smoke results and fresh Postman evidence in **docs/verification-results.json**. It leaves existing application data intact.

Primary references: ASP.NET Core JWT authentication; EF Core migrations; PostgreSQL constraints; Postman Collection Runner.